ngspice-46 vs. Spectre — Feature Gap Analysis

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice-46 vs. a commercial simulator (Spectre) — feature gap analysis

A capability comparison of the ngspice-46 build in this repository against a Spectre-class commercial simulator, to show where the open-source tool already matches the commercial one and where the gaps are. Grounded in a direct read of the `ngspice-46/src/` tree (analyses, solver, devices, RF, convergence, parallelism), not marketing feature lists.

Legend: ✓ present / on par · ⚠ partial, experimental, or via a workaround · × absent. "Spectre" stands in for the commercial reference (Spectre / SpectreRF / Spectre X / RelXpert feature set).

The one column that matters is ngspice — Spectre has essentially everything, so its column is a baseline of ✓. The point of the table is where ngspice's mark is ⚠ or ×.

> Context: the Verilog-A / OSDI device side is not a gap — thanks to the OpenVAF-reloaded compiler in this repository it is on par with (often ahead of) commercial Verilog-A support. The gaps below are all on the simulator/analysis side.

Core numerics

Category	Feature	ngspice	Spectre
Core solver	Sparse LU (KLU) direct solver	✓[1]	✓
Core solver	Legacy Sparse1.3 fallback	✓	✓
Core solver	Parallel / partitioned / GPU linear solve	×	✓
Core solver	Matrix reordering + scaling beyond KLU defaults	⚠	✓
Integration	Trapezoidal + variable-order Gear	✓	✓
Integration	Advanced LTE-based step/order control	✓	✓
Convergence	gmin stepping + source stepping homotopy	✓	✓
Convergence	Pseudo-transient / dynamic-gmin continuation	✓	✓
Convergence	Damped / trust-region (globalized) Newton	⚠	✓
Convergence	Coordinated accuracy presets (`errpreset`)	✓	✓

The Integration "advanced LTE-based step/order control" row was ⚠ because, although ngspice implements Gear coefficients for orders 1–6 (`NIcomCof`) and an LTE-limited-step estimate at any order (`CKTtrunc`/`CKTterr`), the stock transient controller in `dctran.c` only ever toggles the order between 1 and 2 — orders 3–6 were dead code on every ordinary run. Enhancement-128 closes it with `.option dynorder`: each step it compares the raw (uncapped) LTE-limited step at the current order and its ±1 neighbours and moves — with hysteresis, a settling hold, and an order-dependent growth cap — toward the order the local error rewards, so the higher orders are actually used. Off by default, bounded by `maxord`, verified under both linear solvers: 3–5× fewer timesteps at matched accuracy on a smooth RC decay and 8.9× (and more accurate) on a smooth RLC ringdown, while a nonlinear switching circuit is left result-neutral to 5 significant figures.

These two convergence rows: ngspice's DC path is not naked — it has per-device junction limiting (30 device families), an optional `nodedamping` step clamp, and a multi-stage homotopy cascade (`dynamic_gmin` → `new_gmin` → `spice3_gmin`

→ *source stepping). What it lacked vs. commercial tools was a* principled *globalized Newton (residual-based trust-region / Armijo) and a classic pseudo-transient (Ẋ-embedded) continuation — both now added. Enhancement-111 adds the former:* `.option linesearch`*, an Armijo backtracking line search on a new KCL-residual merit* `‖F‖=‖G·x−b‖` *(result-neutral, off by default) — see the implementation write-up. Enhancement-112 then extended it to the KLU solver — it originally ran only under Sparse 1.3 and segfaulted under* `.option klu` *— so the line search now works under both linear solvers, with a residual-merit sequence numerically identical between them. Enhancement-127 adds the latter:* `.option ptcont`*, a pseudo-transient continuation that embeds* `f(x)=0` *in a backward-Euler pseudo-transient* `f(x)+Gps·(x−x_prev)=0` *and marches the pseudo-timestep from small to large (*`Gps→0`*). The* `Gps·x_prev` *RHS coupling makes each step follow a stable trajectory (vs a static gmin step), so it converges — and to the physically correct root — on stiff circuits where plain Newton overshoots; result-neutral and off by default, verified under both linear solvers.*

[1] *KLU is compiled in (SuiteSparse, statically linked) but is not the default in this build — the default direct solver is Sparse 1.3; KLU is selected with* `.option klu`*. The two agree on DC / AC / transient, and — since Enhancement-113 — on noise and single-ended pole-zero as well (the KLU adjoint solve was doing a non-transposed solve, silently wrong on asymmetric matrices; now fixed), and — since Enhancement-114 — on DC/AC sensitivity (*`.sens`*), and — since Enhancement-115 — on distortion (*`.disto`*) too. The only analysis still Sparse-only under KLU is balanced-output pole-zero; KLU is also less robust on stiff transient edges. Full behavior, defaults, and a solver-by-solver sweep of the example suite: KLU vs. Sparse 1.3 solver notes.*

Standard analyses (analog)

Category	Feature	ngspice	Spectre
Analyses	DC operating point / DC sweep	✓	✓
Analyses	Transient (`.tran`)	✓	✓
Analyses	AC small-signal (`.ac`)	✓	✓
Analyses	Noise, small-signal (`.noise`, onoise/inoise + integrated)	✓	✓
Analyses	Pole-zero (`.pz`)	✓	✓
Analyses	Transfer function (`.tf`)	✓	✓
Analyses	DC sensitivity (`.sens`)	✓	✓
Analyses	Distortion (`.disto`)	✓	✓
Analyses	Measurement / post-processing (`.meas`)	✓	✓

RF / periodic steady-state suite

Category	Feature	ngspice	Spectre
RF	S-parameter analysis + noise figure (`.sp`)	✓	✓
RF	Touchstone (`.sNp`) import / export	✓	✓
RF	Periodic steady state (PSS)	✓[1]	✓
RF	Harmonic Balance (HB)	✓[7]	✓
RF	Periodic / phase noise (Pnoise)	⚠[3]	✓
RF	Periodic AC (PAC, conversion gain)	✓[2]	✓
RF	Periodic transfer function (PXF)	✓[4]	✓
RF	Periodic S-parameters (PSP)	✓[5]	✓
RF	Quasi-periodic / multi-tone (QPSS / QPAC)	⚠[6]	✓
RF	Envelope following	×	✓

[1] *PSS was ⚠(experimental* `--enable-pss` *flag, so* `.pss` *was unimplemented in shipped builds, and ~230 lines of shooting-loop trace per run). Since Enhancement-117 it is built by default, quiet (trace behind* `set ngdebug`*), and verified against the analytic AC response; Enhancement-118 then made it run under both linear solvers (KLU had hung on a timestep explosion from reused refactor pivots — now a full re-factor is forced each PSS step under KLU). It is still a brute-force shooting method and remains the foundation for the periodic small-signal analyses below. HB is now implemented -- see note 7 (it had shipped only as a* `WITH_HB` *stub).*

[2] *PAC is ✓ (complete periodic-AC analysis). The full chain is built and verified: Enhancement-119 retains the periodic operating point, Enhancement-120 extracts the periodic Jacobian harmonics* `G_k`, `C_k`*, Enhancement-121 assembles and solves the* `(2M+1)N` *harmonic conversion matrix* `H_{nm}=G_{n−m}+jω_m·C_{n−m}`*, Enhancement-122 wraps it in a user-facing* **`.pac`** *command (runs PSS, sweeps the input frequency, writes a complex plot), and Enhancement-123 finishes it with a netlist-referenced small-signal source stimulus (a true transfer / conversion gain) and multi-sideband output*

— every conversion sideband `f_in + k·f0` as its own named vector (`<node>_usb<k>`/`<node>_lsb<k>`). Verified on the RC: unit-current sideband-0 reproduces the AC driving-point impedance and, with `V1 AC 1`, sideband-0 reproduces the low-pass transfer `1/√(1+(2πfRC)²)` across the band while the conversion sidebands sit at floating-point zero (a linear circuit does not mix). The one scale caveat: the conversion matrix is solved densely (capped for modest circuits) on the brute-force shooting PSS. The same solve is the substrate for pnoise and PXF.

[3] *Pnoise is ⚠(working, stationary-source first cut). Enhancement-124 adds a `.pnoise` command that folds every device's noise through the adjoint of the conversion matrix (`Hᵀ Ψ = e_{out,0}`): it loads the sideband-k transfer into `CKTrhs/CKTirhs` and calls the existing device noise routines (`NevalSrc`, OSDI `load_noise`) once per sideband, accumulating `Σ_k S·|ΔΨ_k|²` — so it reuses every device noise model and needs no per-device code. Verified on the RC: because the circuit is linear the conversion matrix is block-diagonal, only sideband 0 contributes, and pnoise reduces exactly to `.noise` (`4kTR/(1+(2πfRC)²)`, matching the `.noise` reference to every printed digit). Enhancement-126 then adds a cyclostationary mode (`.pnoise … cyclo`): it evaluates each device's noise PSD at* every *PSS sample's bias and folds it through the* time-domain *adjoint transfer, averaging over the period, so a pumped device's bias-dependent noise (a diode's `2qI(t)`, a resistor's flicker `∝|I(t)|²`) is captured correctly. It reduces exactly to the stationary case (and hence `.noise`) for a bias-independent source by Parseval, and on a flicker resistor carrying a known periodic current it gives `onoise·f = R1²·KF·⟨I²⟩` using the period-average `⟨I²⟩` (matched to five digits). It stays ⚠only because it does not yet emit a dedicated phase-noise (jitter) spectrum — a refinement of the same cyclostationary fold.*

*[4] PXF is ✓ (complete). Enhancement-125 adds a `.pxf` command — the adjoint of PAC. It solves `Hᵀ Ψ = e_{out,0}` per frequency and dots each sideband block of `Ψ` with the netlist AC-source pattern `B0` to get the input→output transfer at each sideband (`xf`, `xf_usb<k>`, `xf_lsb<k>`). By the identity `(H⁻¹B)_out = (H⁻ᵀe_out)ᵀB` the sideband-0 transfer is bit-identical to the PAC response at the output — a reciprocity cross-check that pins the adjoint solve — verified on the RC to equal the analytic low-pass transfer, with the conversion sidebands at floating-point zero. The same dense-solve scale caveat as PAC applies. This completes the PSS → PAC → Pnoise → PXF periodic small-signal suite.

[5] *PSP is ✓ since Enhancement-132: a `.psp` command computes periodic small-signal S-parameters by exciting each RF port (`portnum/z0`, the `.sp` framework) through the same `(2M+1)N` conversion matrix and forming `S^(k) = B^(k)·A^−1` per input frequency and conversion sideband `f_in + k·f0`. `pac_solve_at`'s matrix assembly is factored into a shared `pac_build_matrix`; the per-port solve drives each port's branch source (V=1) like `.sp`'s `VSRCspupdate`, and the power waves use the same Kurosawa convention, so `S = B·A^−1` is basis-invariant and reduces exactly to `.sp` for a time-invariant network. Verified 8/8: sideband-0 matches `.sp` to ~10^{-16} for 1/2/3-port resistive and reactive networks (magnitude and phase) — including OSDI Verilog-A devices (conductance and reactive `ddt` stamps) — with correctly-zero conversion sidebands. Runs under both linear solvers (the conversion matrix is a standalone dense LU; PSS runs under both since E-118), verified bit-identical under KLU and Sparse.*

[6] *QPSS is ⚠(working two-tone, transient-sampling) since Enhancement-133: a `qpss` command computes the two-tone steady-state spectrum — every mixing product `k1·f1+k2·f2` including IM3 — for commensurate tones (common beat `fb=gcd(f1,f2)`). It runs an ordinary transient over a few beat periods, then evaluates the Fourier coefficient directly at each exact intermod frequency (no FFT-bin rounding) and labels it by the 2-D index `(k1,k2)`. Solver-independent (drives a transient), works with built-in and OSDI devices. Verified 7/7 checks incl. the analytic IM3 products and the 3:1 IP3 slope law. Still ⚠(not ✓) because the general case wants a true frequency-domain harmonic-balance engine: incommensurate tones (irrational ratio, no common period) are out of reach of transient sampling, and small-signal QPAC is not yet built. HB itself remains a `WITH_HB` stub.*

[7] HB is ✓ since Enhancement-134: a `hb <f0> <K>` command solves the periodic steady state in the frequency domain by Newton -- each node voltage a truncated Fourier series, the KCL residual ‘F_k = I_R,k(V) + [dq/dt]_k

- *Is_k = 0driven to zero with the E−121(2K+1)Nconversion matrix as the exact Jacobian. The device residual/Jacobian are sampled by driving DC+AC loads at the current iterate's voltages; nonlinear **reactive** elements need NO charge extraction becausedq/dt = C(v)v′−− the reactive current is the conversion matrix'sjwCterm applied to V, using the sampledC(t). **Solver−independent (KLU + Sparse):** the dense complex Newton is HB's own, so the linear solver is used only to *read* G(t)/ C(t)off the device matrix −−hb_extractcarries the same#ifdef KLUcomplex−CSC binding as the PAC extraction, andhbhonours.option klu, verified bit−identical under both. Built−in and OSDI devices. Verified 8/8 against the transient/fouriersteady state, with quadratic Newton convergence: nonlinear R (analytic 3rd harmonic), nonlinear R+C, a real **diode rectifier** (junction devices are settled per sample so their limiter is a no−op), a compiled OSDI varactor whoseQ(v)‘ 2nd harmonic matches, and KLU==Sparse parity. Single-tone; strongly-driven continuation, a sparse block solve, and multi-tone HB (true incommensurate QPSS, cf. note 6) are the follow-ups.*

Performance & scale

Category	Feature	ngspice	Spectre
Performance	Multithreaded device-model evaluation (OpenMP)	✓	✓
Performance	Element bypass / latency exploitation	⚠	✓
Performance	Fast-SPICE hierarchical / isomorphic engine	×	✓
Performance	Distributed (MPI) / cloud partitioning	×	✓
Performance	Handles 10^6–10^8-node post-layout netlists	×	✓

Statistical / variability / yield

Category	Feature	ngspice	Spectre
Statistical	Monte Carlo	⚠	✓
Statistical	Native process/mismatch modeling + correlations	⚠	✓
Statistical	Low-discrepancy sampling (Sobol / Latin-hypercube)	×	✓
Statistical	High-sigma methods (importance sampling, worst-case distance)	×	✓
Statistical	Corner + MC + yield estimation flow	⚠	✓

Monte Carlo is ⚠: works, but script-driven (`alter` + `sgauss`, with the deterministic-seed RNG from Enhancement-10/66) rather than a native statistical block with sampling controls.

Reliability / aging

Category	Feature	ngspice	Spectre
Reliability	Device aging (HCI / NBTI / TDDB)	×	✓
Reliability	Stress → degrade → re-simulate (fresh/aged) flow	×	✓
Reliability	Electromigration + IR-drop (EMIR)	×	✓

Post-layout / parasitics

Category	Feature	ngspice	Spectre
Post-layout	Flat parasitic (RC) netlist simulation	✓	✓
Post-layout	RC reduction / model-order reduction	×	✓
Post-layout	n-port (S/Y/Z) extracted-block import	✓	✓

Behavioral / mixed-signal / AMS

Category	Feature	ngspice	Spectre
Modeling	Verilog-A (analog, LRM Annex C) via OSDI	✓	✓
Modeling	XSPICE code models + event-driven digital	✓	✓
Modeling	Verilog-AMS mixed-signal (connect modules)	×	✓
Modeling	Real-number modeling (wreal / RNM)	×	✓
Modeling	VHDL-AMS	×	✓

Interfaces & infrastructure

Category	Feature	ngspice	Spectre
Interface	Shared-library / programmatic API	✓	✓
Interface	Python bindings	✓	✓

Category	Feature	ngspice	Spectre
Infrastructure	Built-in optimizer	✓	✓
Infrastructure	Checkpoint / restart of long runs	✓	✓

The "built-in optimizer" row is ✓ since Enhancement-130: the `optimize` command is a derivative-free Nelder-Mead search that varies device/`alter` parameters, re-runs an analysis, and minimizes an objective expression in normalized [0,1] space — verified to reach analytic optima in 1-D and 2-D.

The "checkpoint / restart" row is ✓ since Enhancement-131: stock ngspice could only continue a paused run in memory (`stop`/`resume`); the new `savestate <file>` / `loadstate <file>` commands serialize the full transient integration state (solution vector, device state history, time/step/order, pending breakpoints) to disk and resume it — including in a fresh process — so a long run survives a crash, splits across sessions, or moves between machines. The resumed waveform is bit-identical to an uninterrupted run for built-in devices (Sparse solver).

Where to invest (given the Verilog-A/OSDI side is done)

The realistic, winnable identity for this project is the open-source, OSDI/Verilog-A-native simulator with a real RF-noise and statistical story — not out-engineering 25 years of commercial fast-SPICE/parallel work. Ranked by leverage on the existing strength × differentiation × tractability:

1. RF periodic small-signal suite — PAC → Pnoise → PXF, on the hardened PSS. The PSS foundation is now shipped and verified under both linear solvers (Enhancement-117, Enhancement-118), and Enhancement-119 retains the periodic operating point (voltages + device states per sample) and Enhancement-120 turns it into the periodic Jacobian harmonics `G_k`, `C_k`, and Enhancement-121 assembles those into the `(2M+1)N` harmonic conversion matrix and solves it, Enhancement-122 wraps that in a working **`.pac`** command, and Enhancement-123 finishes it with a netlist-referenced source stimulus and multi-sideband conversion-gain output — a complete periodic-AC analysis, verified against the exact linear transfer and driving-point responses, Enhancement-124 adds **`.pnoise`** — folding every device's noise through the conversion-matrix adjoint H^T (reusing the existing device noise routines), verified to reduce exactly to `.noise` in the linear limit — and Enhancement-125 adds **`.pxf`**, the adjoint transfer function, whose sideband-0 result is bit-identical to the PAC response by reciprocity. The PSS → PAC → Pnoise → PXF periodic small-signal suite is now complete — genuinely novel in open source — and Enhancement-126 adds cyclostationary noise (per-sample bias, time-domain-transfer fold) so pumped devices' bias-dependent noise is handled correctly. The remaining RF work is the further refinements (a dedicated phase-noise/jitter spectrum, a from-scratch Harmonic Balance engine, quasi-periodic multi-tone) rather than the core analyses.
2. Convergence robustness — coordinated accuracy presets (`errpreset`) landed in Enhancement-110, a globalized Newton line search in Enhancement-111/112, and pseudo-transient continuation (`.option ptcont`) in Enhancement-127 — so the principled globalization and continuation gaps are now closed. What remains is mostly auto-triggering heuristics (reaching for these aids without the user asking) and folding them into the robustness presets. Self-contained in the ngspice core.
3. High-sigma statistical sampling — high industrial value (yield / SRAM), moderate difficulty, leans on the deterministic-seed RNG already in place.

Explicitly lower priority: fast-SPICE parallelism and aging/EMIR — enormous efforts that don't leverage what makes this project distinctive.